TP Algorithmique n°1 — Corrigé

Premiers pas en Python
affectation et fonctions

Objectifs de la séance.
Découvrir Capytale, l'outil en ligne dans lequel nous programmerons en Python ; revoir ce que nous avons déjà rencontré dans les automatismes (variables, affectation, fonctions) ; écrire, tester et corriger ses premiers programmes.

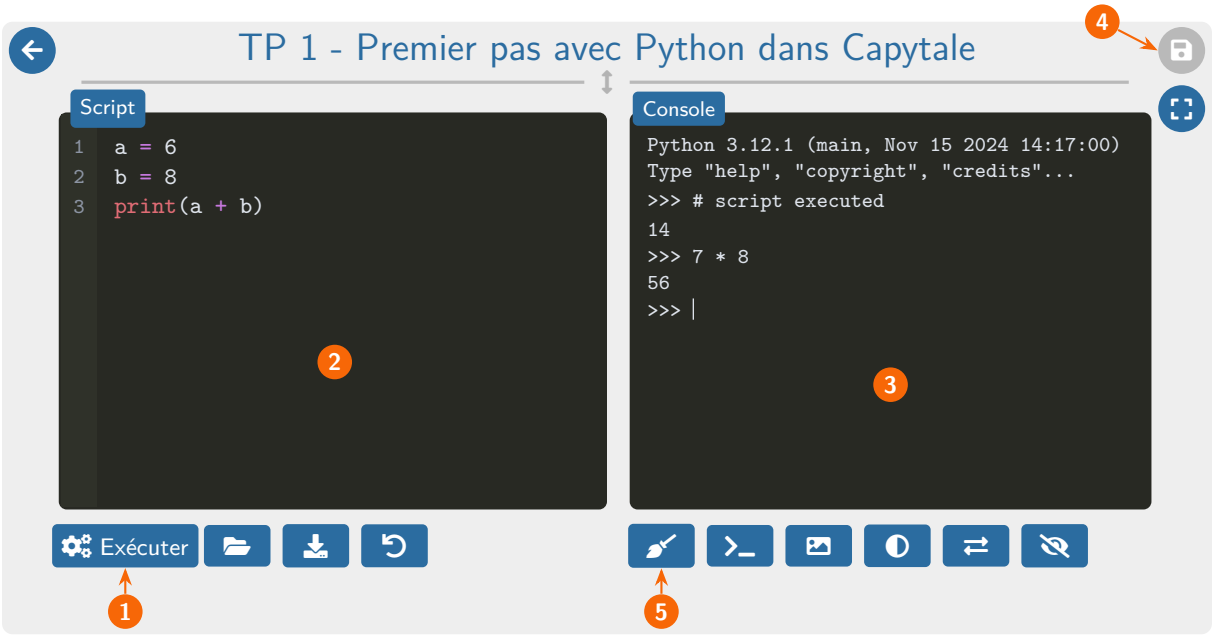
Partie 1 — Découvrir Capytale

Capytale est un service de l'Éducation nationale qui permet d'écrire et de faire fonctionner des programmes Python **directement dans le navigateur**, sans rien installer. Le professeur crée une activité ; chaque élève travaille sur **sa propre copie**, que le professeur peut consulter.

① Se connecter à l'ENT, puis ouvrir **Capytale**.

② Saisir le **code de l'activité** : `0d37-11887835`

③ L'activité s'ouvre sur l'écran ci-dessous.



1. Le bouton **Exécuter**, sous le script, lance le programme de la première à la dernière ligne.
2. Le **Script** (éditeur) : on y écrit le programme. Il est conservé, on peut le modifier et le relancer autant de fois qu'on veut.
3. La **Console** : après un clic sur Exécuter, elle affiche `# script executed` (« script exécuté »), puis les résultats du script. Après le symbole >>>, on peut aussi taper une instruction : Python répond aussitôt (touche Entrée).
4. La **disquette**, en haut à droite, enregistre le travail : à faire régulièrement.
5. Le **balai** efface le contenu de la console.

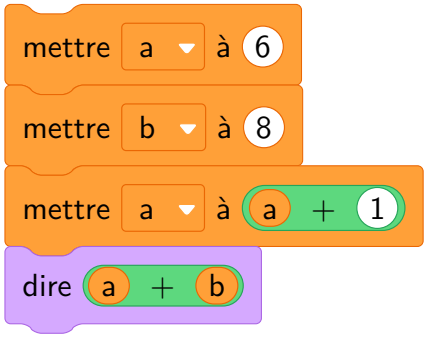
Les règles d'or du programmeur débutant

- ⋆ Python lit **exactement** ce qui est écrit : un « : » oublié, une parenthèse non fermée, une majuscule à la place d'une minuscule… et c'est l'erreur.
- ⋆ un nombre à virgule s'écrit avec un **point** : `2.5` (et non `2,5`). La multiplication s'écrit toujours : `3 * x` (et non `3x`).
- ⋆ un message d'erreur en rouge n'est pas une catastrophe : il indique la **ligne** en cause (`line 2`) et le **type** d'erreur. On le lit, on corrige, on relance.

Partie 2 — Ce que nous avons déjà vu dans les automatismes

Rappel 1 : variables et affectation

En Scratch



En Python

Script

```
1 a = 6
2 b = 8
3 a = a + 1
4 print(a + b)
```

Console

```
>>> # script executed
15
```

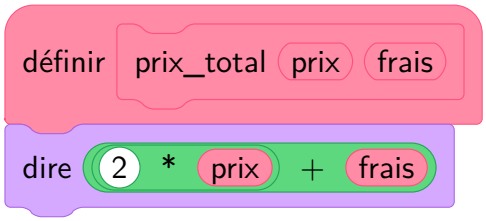
À retenir :

- `a = 6` se lit « a prend la valeur 6 » : ce n'est pas une égalité mathématique, c'est une **affectation**.
- Dans `a = a + 1`, Python calcule d'abord `a + 1` avec l'ancienne valeur ($6 + 1 = 7$), puis range le résultat dans `a`.
- `print(...)` affiche une valeur dans la console.
- le **nom** d'une variable ne contient ni espace ni accent : on remplace les espaces par des tirets bas, par exemple `nouveau_prix`.
- **Nouveau :** l'**affectation simultanée**
  `a, b = 5, 9` affecte *en même temps* la valeur 5 à `a` et la valeur 9 à `b` : c'est la même chose que `a = 5` puis `b = 9`, mais en une seule ligne.

Opération	addition	soustraction	multiplication	division	puissance
En maths	$7+2$	$7-2$	7×2	$7\div 2$	7^2
En Python	`7 + 2`	`7 - 2`	`7 * 2`	`7 / 2`	`7 ** 2`

Rappel 2 : les fonctions

En Scratch (bloc personnalisé)



En Python

Script

```
1 def prix_total(prix, frais):
2     return 2 * prix + frais
```

Console

```
>>> prix_total(5, 4)
14
```

À retenir :

- `def` annonce la définition d'une fonction, suivie de son **nom**, de ses **paramètres** entre parenthèses (ici `prix` et `frais`) et de **deux-points** `:`.
- les instructions de la fonction sont **décalées vers la droite** (indentation, 4 espaces ou touche Tab).
- `return` indique la valeur **renvoyée** par la fonction.
- exécuter le script ne fait que *définir* la fonction : pour l'utiliser, on l'**appelle** dans la console avec des valeurs, par exemple `prix_total(5, 4)` : $2\times 5+4=14$.

Partie 3 — Exercices d'application

Exercice 1 ★☆☆

Calculer

La console comme calculatrice. Taper chaque calcul dans la console et noter le résultat affiché.

On tape	`7 * 8`	`2 ** 10`	`3 + 4 * 5`	`(3 + 4) * 5`
Python affiche	**56**	**1024**	**23**	**35**
On tape	`7 / 2`	`10 / 5`	`2.5 * 4`	`1.5 * 6 + 3`
Python affiche	**3.5**	**2.0**	**10.0**	**12.0**

① Python respecte-t-il les priorités opératoires ? Justifier avec deux résultats du tableau.

> Oui : `3 + 4 * 5` donne 23 (la multiplication est faite en premier), alors que `(3 + 4) * 5` donne 35 (les parenthèses sont prioritaires).

② Certains résultats s'écrivent avec « `.0` ». Dans quels cas ?

> Après une division `/` (même si elle « tombe juste », comme $10 \div 5$), ou dès qu'un nombre à virgule intervient dans le calcul. Python donne alors un *nombre à virgule* (un « flottant »), même si sa partie décimale est nulle.

③ Taper `3,5 * 2`. Qu'affiche Python ? Comment faut-il écrire le calcul $3{,}5 \times 2$?

> Python affiche `(3, 10)` : pour lui, la virgule *sépare* deux nombres, 3 et $5 \times 2 = 10$. Il faut écrire `3.5 * 2`, qui donne `7.0`.

Exercice 2 ★★☆

Raisonner

Prévoir, puis vérifier.
Pour **chaque** script, **prévoir d'abord** sur la feuille ce qui va s'afficher, puis le recopier dans Capytale et l'exécuter pour vérifier.

Script A

Script

```
a = 5
b = 3
a = a + b
b = a * 2
print(a)
print(b)
```

Script B

Script

```
x = 4
y = 7
x = y
y = x
print(x)
print(y)
```

Script C *(énergie cinétique d'un objet de 50 kg lancé à 12 m/s)*

Script

```
masse = 50
vitesse = 12
energie = 0.5 * masse * vitesse ** 2
print(energie)
```

	Je prévois	**Python affiche**
Script A	**8 puis 16**	**8 puis 16**
Script B	**7 puis 7**	**7 puis 7**
Script C	**3600.0**	**3600.0**

Script A : a vaut $5+3=8$, puis b vaut $8\times 2=16$ (on utilise la *nouvelle* valeur de a).
Script B : après `x = y`, `x` vaut 7 ; l'ancienne valeur 4 est perdue. Donc `y = x` donne à `y` la valeur 7.
Script C : la puissance est prioritaire, seule la vitesse est élevée au carré : $0{,}5\times 50\times 12^2 = 0{,}5\times 50\times 144 = 3\,600$.
Python affiche `3600.0` car `0.5` est un nombre à virgule.

① Le script B devait **échanger** les valeurs de `x` et `y`. Est-ce réussi ? Expliquer.

Non : les deux variables valent 7 à la fin. Au moment de `y = x`, `x` a déjà été écrasée par la valeur de `y`.

② Corriger le script B en utilisant une troisième variable `z`, puis le vérifier dans Capytale.

Script

```
x = 4
y = 7
z = x      # on met de côté la valeur de x
x = y
y = z
print(x)   # affiche 7
print(y)   # affiche 4
```

③ On utilise maintenant l'affectation simultanée (voir le rappel 1).
Taper dans la console, l'une après l'autre, les instructions ci-dessous. Que fait l'instruction `x, y = y, x` ?

Console

```
>>> x, y = 4, 7
>>> x, y = y, x
>>> x
>>> y
```

`x, y = 4, 7` affecte 4 à `x` et 7 à `y`. `x, y = y, x` échange les deux valeurs en une seule instruction : la console affiche ensuite 7 puis 4. Python calcule d'abord toutes les valeurs à droite du =, avant de les ranger : c'est pourquoi l'ancienne valeur de `x` n'est pas perdue.

Exercice 3 ★☆☆

Chercher

Chasse aux erreurs.
Voici une fonction censée renvoyer le triple d'un nombre. Elle contient **trois erreurs**.

Script

```
def triple(x)
return 3x
```

① Recopier ce script tel quel et l'exécuter. Lire le message d'erreur, corriger **une** erreur, relancer, et ainsi de suite jusqu'à ce que `triple(5)` renvoie 15 dans la console. Compléter le tableau.

	Message d'erreur (dernière ligne)	**Correction apportée**
1	`SyntaxError: invalid decimal literal`	3x devient `3 * x`
2	`SyntaxError: expected ':'`	ajouter : à la fin de la ligne 1
3	`IndentationError: expected an indented block...`	décaler `return` vers la droite

Python signale les erreurs **une par une** : c'est pour cela qu'il faut relancer après chaque correction. Script corrigé :

Script

```
1 def triple(x):
2     return 3 * x
```

② Taper maintenant `Triple(5)` dans la console. Que se passe-t-il ? Pourquoi ?

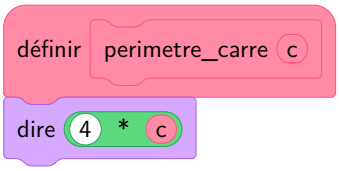
`NameError: name 'Triple' is not defined` : Python distingue majuscules et minuscules. La fonction s'appelle `triple`, pas `Triple`.

Exercice 4 ★☆☆

Représenter

De Scratch à Python.

① Voici un bloc Scratch qui calcule le périmètre d'un carré de côté c.



Écrire dans Capytale la fonction Python `perimetre_carre` correspondante, puis compléter :

`perimetre_carre(7)` renvoie **28** `perimetre_carre(2.5)` renvoie **10.0**

Script

```
1 def perimetre_carre(c):
2     return 4 * c
```

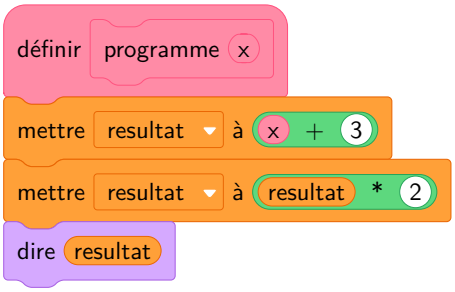
② Écrire une fonction `aire_rectangle` à **deux** paramètres `L` et `l` qui renvoie l'aire d'un rectangle de longueur L et de largeur ℓ. Compléter :

`aire_rectangle(5, 3)` renvoie **15** `aire_rectangle(2.5, 4)` renvoie **10.0**

Script

```
1 def aire_rectangle(L, l):
2     return L * l
```

③ Voici un programme de calcul écrit en Scratch.



a) Écrire la fonction Python `programme(x)` correspondante, en utilisant une variable `resultat`.

b) `programme(4)` renvoie **14**

c) `programme(-3)` renvoie **0**

Script

```
1 def programme(x):
2     resultat = x + 3
3     resultat = resultat * 2
4     return resultat
```

$(4+3)\times 2 = 14$ et $(-3+3)\times 2 = 0$. On pouvait aussi écrire directement `return (x + 3) * 2` ; sans les parenthèses, `x + 3 * 2` calculerait $x + 6$.

Exercice 5 ★☆☆

Modéliser

Vitesse moyenne. On a écrit la fonction suivante.

Script

```
def vitesse(distance, temps):
    return distance / temps
```

① Combien de paramètres cette fonction possède-t-elle ? Lesquels ?

> Deux paramètres : `distance` et `temps`.

② La distance est exprimée en kilomètres et le temps en heures. Dans la console, `vitesse(120, 1.5)` renvoie `80.0`. Que signifie ce résultat ?

> Parcourir 120 km en 1,5 h (1 h 30 min) correspond à une vitesse moyenne de 80 km/h : $120 \div 1{,}5 = 80$.

③ Écrire dans Capytale une fonction `temps(d, v)` qui renvoie le temps (en heures) nécessaire pour parcourir la distance `d` (en km) à la vitesse `v` (en km/h). Compléter :

`temps(300, 80)` renvoie **3.75**, soit une durée de **3 h 45 min**.

Script

```
def temps(d, v):
    return d / v
```

> $300 \div 80 = 3{,}75$ h, et $0{,}75 \text{ h} = 0{,}75 \times 60 = 45$ min.

Exercice 6 ★★☆

Modéliser

Degrés Celsius et degrés Fahrenheit. Aux États-Unis, les températures sont données en degrés Fahrenheit (°F). Pour convertir en °F une température C donnée en degrés Celsius, on utilise la formule :

$$F = 1{,}8 \times C + 32.$$

① Écrire une fonction `fahrenheit(C)` qui renvoie la température en °F.

Script

```
def fahrenheit(C):
    return 1.8 * C + 32
```

② L'utiliser dans la console pour compléter le tableau.

Température en °C	0	100	20	-40
Température en °F	**32.0**	**212.0**	**68.0**	**-40.0**

③ Que remarque-t-on pour $-40\,°C$?

$-40\,°C$ correspond à $-40\,°F$: c'est la seule température qui s'écrit de la même façon dans les deux unités (solution de l'équation $1{,}8C + 32 = C$).

④ En résolvant l'équation $F = 1{,}8 \times C + 32$ d'inconnue C, écrire une fonction `celsius(F)` qui fait la conversion inverse. Vérifier que `celsius(212)` renvoie **100.0**.

$F = 1{,}8C + 32 \iff F - 32 = 1{,}8C \iff C = \dfrac{F - 32}{1{,}8}$. Les parenthèses sont indispensables en Python :

Script

```
def celsius(F):
    return (F - 32) / 1.8
```

Exercice 7 ★★☆

Modéliser

Hausses et remises. On rappelle qu'augmenter une quantité de p % revient à la multiplier par $CM = 1 + \dfrac{p}{100}$, et la diminuer de p % revient à la multiplier par $CM = 1 - \dfrac{p}{100}$.

① Le loyer d'un appartement est de 500 € par mois. Il augmente de 5 %. Compléter le script, puis l'exécuter.

Script

```
loyer = 500
taux = 5
nouveau_loyer = loyer * (1 + taux / 100)
print(nouveau_loyer)
```

Python affiche **525.0** : le nouveau loyer est de **525** €.

$CM = 1 + \dfrac{5}{100} = 1 + 0{,}05 = 1{,}05$ et $500 \times 1{,}05 = 525$. Les parenthèses autour de `1 + taux / 100` sont indispensables.

② Écrire une fonction `solde(prix, remise)` qui renvoie le prix d'un article après une remise de `remise` %. Compléter :

`solde(80, 25)` renvoie **60.0** `solde(49.90, 30)` renvoie **34.93**

Script

```
def solde(prix, remise):
    return prix * (1 - remise / 100)
```

Pour 25 % : $CM = 1 - \dfrac{25}{100} = 1 - 0{,}25 = 0{,}75$ et $80 \times 0{,}75 = 60$.

Exercice 8 ★★★

Raisonner

Le programme mystère. On considère la fonction suivante.

Script

```
def mystere(x):
    return (x + 3) ** 2 - (x + 1) ** 2 - 4 * x
```

① Recopier la fonction dans Capytale, puis compléter le tableau à l'aide de la console.

x	5	-2	0	10,5	1 000
`mystere(x)`	**8**	**8**	**8**	**8.0**	**8**

② Quelle conjecture peut-on faire ?

Quel que soit le nombre x choisi, la fonction semble toujours renvoyer 8.

③ Démontrer cette conjecture en développant et en réduisant l'expression $(x+3)^2 - (x+1)^2 - 4x$.

$(x+3)^2 = (x+3)(x+3) = x^2 + 3x + 3x + 9 = x^2 + 6x + 9$ et $(x+1)^2 = x^2 + 2x + 1$.
Donc $(x+3)^2 - (x+1)^2 - 4x = x^2 + 6x + 9 - (x^2 + 2x + 1) - 4x = x^2 + 6x + 9 - x^2 - 2x - 1 - 4x = 8$.
L'expression vaut 8 pour **tout** nombre x : la conjecture est démontrée. Tester des valeurs ne suffit pas à le prouver ; le calcul littéral, si.

④ *Défi.* Écrire une fonction `mystere2(x)` de la même forme, `(x + a) ** 2 - (x + b) ** 2 - c * x`, avec des nombres a, b et c bien choisis, qui renvoie toujours 20. Vérifier avec la console.

En développant : $(x+a)^2 - (x+b)^2 - cx = (2a - 2b - c)\,x + a^2 - b^2$. Il faut donc $c = 2a - 2b$ (pour que les x disparaissent) et $a^2 - b^2 = 20$. Par exemple $a = 6$, $b = 4$, $c = 4$: $36 - 16 = 20$.

Script

```
def mystere2(x):
    return (x + 6) ** 2 - (x + 4) ** 2 - 4 * x
```

D'autres choix conviennent, par exemple $a = 4{,}5$, $b = 0{,}5$, $c = 8$.